

ARACHNID

Hardware → Software Integration & Mapping Architecture

Detailed handoff document for the software team

Purpose: define exactly how the existing team software should be adapted to the actual two-ESP32 ARACHNID hardware, how sensor data reaches the software, how commands return to the rover, how mapping is produced, and what must be changed before physical integration.

Source basis: the uploaded team software package was inspected, including its ROS2 handoff, configuration files, Python backend, ROS2 provider, adapters, WebSocket layer, and React mapping UI. The actual hardware baseline below is the hardware/firmware established in this project conversation.

1. Executive Summary

The team software is already structured as a command-center application with a Python backend, ROS2 integration layer, WebSocket streaming, and a React frontend. The existing software is therefore reusable; the main task is to replace the assumptions in its hardware interface with the actual ARACHNID interface.

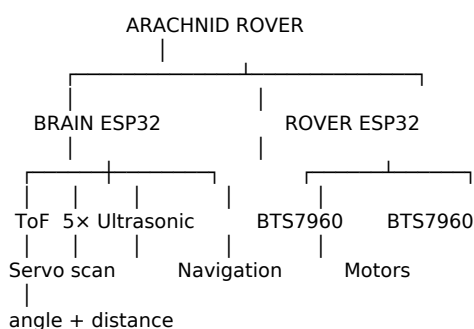
The biggest mismatches are: the software currently models four ultrasonics while the rover has five; the software models the ToF as a single front distance while the actual ToF is mounted on a continuously scanning servo; the software expects wheel encoder/IMU data for localization while the current hardware does not expose those signals; and the software's real provider expects rosbridge topics, while the current ESP32 firmware speaks a simple command protocol and must gain a network telemetry interface if the laptop is to receive data over Wi-Fi.

Recommended integration principle: keep the tested motor direction logic and low-speed PWM on the Rover ESP32, keep the Brain ESP32 responsible for sensor acquisition, and add a clean network data gateway between the Brain and the laptop software. Do not rewrite the React UI or the mapping UI from scratch.

For a full ROS2 autonomous-navigation architecture, the software must eventually become the authority that publishes velocity commands, while the ESP32 layer becomes a sensor/actuator gateway. With the current hardware, however, global persistent SLAM is limited because reliable rover pose (odometry/IMU) is not currently available.

2. Current Hardware Architecture

Actual two-controller architecture:



Brain ESP32 responsibilities:

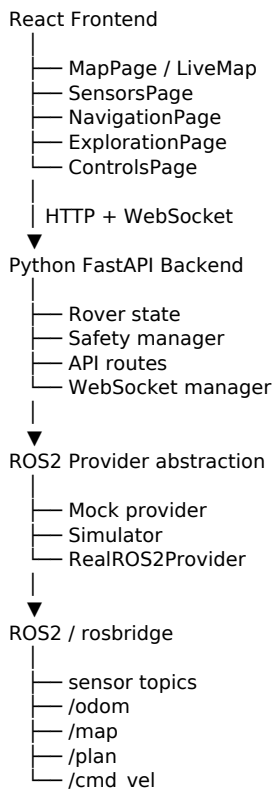
- Read the VL53L0X ToF continuously while the servo scans 30° ↔ 150° in 2° increments.
- Read five ultrasonic channels (currently only the front sensor is physically connected during testing; unconnected channels report the firmware's 999 sentinel).
- Run the current low-level obstacle-avoidance logic during the present prototype phase.
- Send discrete movement commands to the Rover ESP32 over UART: FORWARD, BACKWARD, LEFT, RIGHT, STOP.

Rover ESP32 responsibilities:

- Drive the two BTS7960 channels using the already-tested direction mapping.
- Use a low default motor PWM of 80 for mapping/exploration testing.
- Receive Brain commands over UART and enforce a 1-second command failsafe.
- Provide Wi-Fi/manual-control services in the current firmware; this should be separated from the mapping transport in the final architecture.

3. Actual Software Architecture in the Team Package

The uploaded project already contains the following major layers:

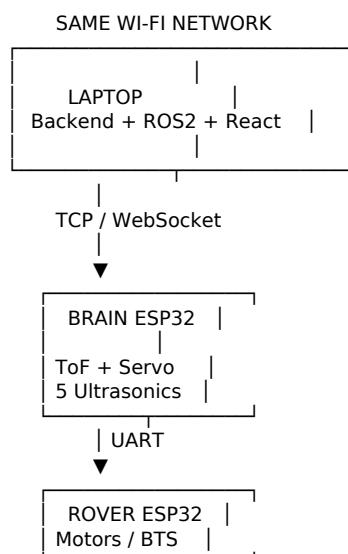


The project's RealROS2Provider currently connects to rosbridge at 127.0.0.1:9090 and subscribes to /sensors/range and /map. It publishes velocity commands to /cmd_vel and emergency-stop messages to /emergency_stop. This means the existing provider is not directly compatible with the current ESP32 ASCII command protocol.

The frontend already consumes WebSocket events such as sensor_telemetry, rover_status, map_update, navigation_update and exploration_update. Therefore, the frontend should remain largely unchanged; the backend/gateway should normalize hardware data into the existing frontend contract.

4. Recommended End-to-End Connection

For the physical demo, use one common Wi-Fi network so the laptop software and the Brain ESP32 can reach each other. The simplest operational topology is a router or laptop/mobile hotspot; both the laptop and Brain join the same LAN. The Rover can remain connected to the Brain through its existing UART link.



Important: the current Rover and Brain sketches were designed around ESP32 access-point/manual-control behavior. For the integrated software architecture, the Brain should expose a stable TCP/WebSocket telemetry endpoint, and the laptop should connect to that endpoint. Avoid having both ESP32s independently create an AP with the same SSID; the integrated system should have one clear network authority.

5. Data Flow — Sensors to Software

The actual ToF data is already being produced as angle/distance pairs such as MAP,118,38. That is the correct raw information for a scanning range sensor.

ToF measurement:
servo_angle = 118 deg
distance = 38 cm

Raw record:
MAP,118,38

Recommended network JSON:
{
 "type": "tof_scan",
 "timestamp_ms": 1720000000000,
 "angle_deg": 118,
 "distance_mm": 380
}

The gateway should convert all raw ESP32 values to one canonical unit: millimetres for sensor ranges. Invalid measurements should be represented as -1 internally, matching the team's interface contract, rather than forwarding 999 as if it were a real range.

Recommended combined telemetry envelope:

```
{  
  "type": "sensor_telemetry",  
  "timestamp_ms": 1720000000000,  
  "seq": 1234,  
  "ultrasonic": {  
    "front_mm": 310,  
    "right_mm": -1,  
    "left_mm": -1,  
    "front_right_45_mm": -1,  
    "front_left_45_mm": -1  
  },  
  "tof_scan": {  
    "angle_deg": 118,  
    "distance_mm": 380  
  }  
}
```

The frontend does not need to know the ESP32 protocol. The backend/gateway owns the conversion from hardware protocol to the software contract.

6. Exact Software Changes Required

A. hardware.yaml — change the hardware description

```
robot:  
  wheel_diameter_m: <MEASURE>  
  track_width_m: <MEASURE>  
  
sensors:  
  ultrasonic:  
    count: 5  
    names:  
      - us_front  
      - us_right  
      - us_left  
      - us_45_right  
      - us_45_left  
    max_range_mm: <CONFIGURE>  
    update_hz: 10
```

```

tof:
  enabled: true
  model: VL53L0X
  max_range_mm: 2000
  update_hz: 20

imu:
  enabled: false

encoders:
  enabled: false

network:
  use_wifi: true
  wifi_ip: <BRAIN_IP>
  wifi_port: <BRAIN_PORT>

```

B. interfaces.yaml — change telemetry model

- Replace the current four-ultrasonic model with five named ultrasonic channels matching the actual Brain firmware.
- Retain invalid_value: -1 as the software representation of disconnected/invalid readings.
- Add a tof_scan representation containing angle_deg and distance_mm, not only tof_front_mm.
- Mark IMU and encoders as unavailable until actual telemetry is implemented.

C. adapters.py — change SensorAdapter

- Accept the Brain's network JSON instead of assuming only us_fl/us_fr/us_l/us_r/tof_front_mm.
- Convert centimetres to millimetres if the Brain transmits cm.
- Map 999/no-echo to -1 and status DISCONNECTED or TIMEOUT.
- Validate ultrasonic ranges against 20–4000 mm and ToF against 10–2000 mm.
- Store the latest ToF scan point and optionally a short rolling scan buffer for the current sweep.

D. real_provider.py — do not force the ESP32 into the current rosbridge contract immediately

The current RealROS2Provider is explicitly a rosbridge client. For first physical integration, add a second provider or gateway, for example ESP32Provider, that connects directly to the Brain's WebSocket/TCP endpoint. Keep RealROS2Provider intact for ROS2 simulation/ROS2 deployments.

Provider selection:

```
ARACHNID_ROS2_PROVIDER=esp32
```

ESP32Provider:

```

connect to BRAIN_IP:BRAIN_PORT
receive sensor_telemetry / tof_scan
normalize data
update RoverStateManager
broadcast sensor_telemetry to WebSocketManager
expose safe command path

```

E. rover_state.py / schemas.py

- Add scan state or a scan buffer if the UI needs the live polar scan.
- Keep existing RoverStatus, Telemetry, MapData, NavigationData and ExplorationData contracts where possible.
- Do not report the robot as fully localized unless pose data is actually available.

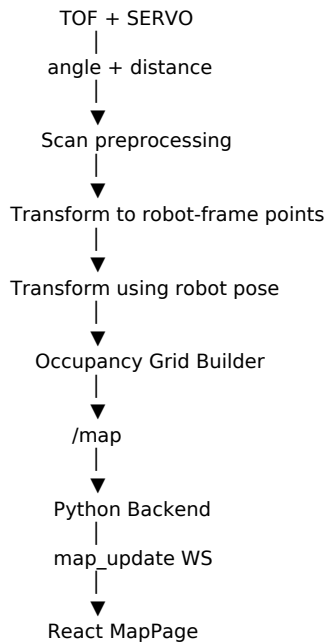
F. frontend/src/services/websocket.ts and useRover.tsx

- Keep the existing reconnect/staleness mechanism.
- Add a subscription for a new tof_scan or sensor_scan event if the UI will display the live rotating scan.
- Keep map_update unchanged if the backend emits the existing MapData format.

- Do not put ESP32 IP addresses into React components; use the backend URL only.

7. Mapping Architecture

The team's MapPage already renders an occupancy grid from MapData. The map renderer therefore does not need to be replaced. What must be supplied is a correct /map data stream.



For a single scan around a stationary rover, the conversion is straightforward:

```

x_sensor = distance_m * cos(angle_rad)
y_sensor = distance_m * sin(angle_rad)
  
```

For a global room map while the rover is moving, each point must additionally be transformed using the rover's current x, y and heading. Therefore a true persistent map requires a reliable pose estimate.

The current software handoff expects /odom at 20 Hz, while its hardware configuration also declares encoders and IMU. The current hardware/firmware described in this project does not currently provide encoder/IMU pose telemetry. This is the main technical blocker to reliable global SLAM/localization.

8. What Happens to the Existing Navigation Code?

There are two possible integration modes. Choose one explicitly; do not let both the Brain and ROS2 navigation independently command the rover.

MODE A — Prototype / safest immediate integration

- Brain remains the navigation authority.
- Brain reads ultrasonics + ToF and sends FORWARD/LEFT/RIGHT/BACKWARD/STOP to the Rover.
- Software team receives telemetry and displays status/scan/map information.
- ROS2 navigation remains disabled or used only for visualization/simulation.
- Advantage: preserves the currently tested motor firmware and Brain navigation.

MODE B — Full software/ROS2 autonomy

- ROS2/Nav2 becomes the only navigation authority.
- Brain becomes a sensor gateway rather than making final movement decisions.
- Rover becomes an actuator gateway that receives velocity commands from the software stack.
- The software's /cmd_vel Twist contract must be translated into motor commands.

- Reliable odometry/pose must be supplied for mapping/localization.
- Do not simultaneously let Brain autonomy and Nav2 publish competing commands.

For the current hardware state, Mode A should be the first milestone. Mode B is the target architecture once pose/actuation integration is ready.

9. Velocity Command Mismatch

The team's software defines `/cmd_vel` as linear.x in m/s and angular.z in rad/s. The current Rover firmware accepts only discrete ASCII commands. These are not equivalent.

Team software:

```
linear_mps = 0.15
angular_rads = 0.30
```

Current Rover firmware:

```
FORWARD
BACKWARD
LEFT
RIGHT
STOP
```

If the team insists on using Nav2 immediately, a gateway can perform a crude command quantization, but this will reduce navigation quality because the current Rover firmware does not expose independent left/right wheel velocity control. The preferred long-term actuator interface is a velocity command protocol that the Rover firmware can translate to left/right wheel speeds.

Until that actuator interface exists, keep autonomous movement authority on the Brain and use the software stack for monitoring and mapping experiments.

10. Network Integration — Exact Operational Flow

Recommended demo sequence:

- Start the common Wi-Fi network.
- Connect the laptop to that network.
- Power the Brain and Rover.
- Brain joins the network and reports its IP address.
- Run the backend with the ESP32 provider enabled.
- Run the React frontend.
- Open the frontend in the browser.
- Backend connects to the Brain's telemetry endpoint.
- Brain streams ToF scan and ultrasonic telemetry.
- Backend normalizes data and broadcasts `sensor_telemetry/tof_scan` events.
- Frontend displays sensor state and live scan.
- Once pose data is available, mapping publishes `MapData` and the existing `MapPage` renders it.

Do not make the browser connect directly to both ESP32s. The browser should connect to the backend; the backend should own hardware communication. This keeps the architecture secure, testable and independent of the UI.

11. Proposed Network Protocol

Use JSON messages over WebSocket between the Brain and the laptop gateway. One message per telemetry sample is simple and debuggable.

```
{
  "type": "telemetry",
  "timestamp_ms": 1720000000000,
  "seq": 1001,
```

```

"us_front_mm": 310,
"us_right_mm": -1,
"us_left_mm": -1,
"us_45_right_mm": -1,
"us_45_left_mm": -1,
"tof_angle_deg": 118,
"tof_distance_mm": 380
}

```

For commands in Mode A, the gateway should use the current Brain/Rover command vocabulary:

```

{"type":"rover_command","command":"FORWARD"}
{"type":"rover_command","command":"LEFT"}
{"type":"rover_command","command":"RIGHT"}
{"type":"rover_command","command":"BACKWARD"}
{"type":"rover_command","command":"STOP"}

```

For Mode B, introduce a separate velocity command schema only after the Rover firmware supports it.

12. Existing Frontend — What Should Stay

- MapPage.tsx: keep the occupancy-grid renderer. It already consumes MapData and draws map cells, navigation path, goal and rover marker.
- LiveMap.tsx: keep it as the reusable live-map component; feed it the same normalized MapData.
- useRover.tsx: keep the existing state/context architecture; add a scan subscription rather than replacing the WebSocket service.
- websocket.ts: keep reconnect and stale-connection behavior.
- api.ts: keep the existing REST API surface for status, telemetry, map, navigation, exploration and commands.
- ControlsPage / NavigationPage / ExplorationPage: keep the UI, but disable controls that depend on unavailable hardware until their backend contract is implemented.

The frontend should not contain sensor parsing, coordinate transforms, ROS2 logic or motor command translation. Those belong in the backend/integration layer.

13. Software File-by-File Change List

File	Change	Priority
config/hardware.yaml	5 ultrasonics; VL53L0X; current availability of IMU/encoders; actual network settings; safety limits	HIGH
config/interfaces.yaml	Add 5th ultrasonic + ToF scan fields; preserve -1 invalid convention	HIGH
config/commands.yaml	Keep current command contract for Mode A; add explicit mapping plan for Mode B	HIGH
backend/ros2/adapters.py	Normalize ESP32 JSON, cm→mm, 999→-1, validate status	HIGH
backend/ros2/real_provider.py	Leave rosbridge path intact; do not hard-wire it to ESP32	MEDIUM
backend/ros2/esp32_provider.py (NEW)	Direct Brain WebSocket/TCP provider; telemetry + command gateway	HIGH
backend/ros2/factory.py	Add provider selection: esp32	HIGH
backend/models/schemas.py	Add ToF scan model/event if required	HIGH
backend/services/rover_state.py	Store latest telemetry/scan and connection state	HIGH
backend/websocket/manager.py	Broadcast normalized scan events	MEDIUM
frontend/src/hooks/useRover.tsx	Subscribe to tof_scan; retain existing map/status subscriptions	MEDIUM
frontend/src/services/websocket.ts	No architecture rewrite; support new event type	LOW
frontend/src/components/LiveMap.tsx	Only modify if displaying raw scan overlay	LOW
frontend/src/pages/MapPage.tsx	Keep occupancy grid; optionally add live scan overlay	LOW
frontend/src/config/index.ts	Point UI to backend host; do not hard-code ESP32 IP	HIGH
docs/ROS2_HARDWARE_HANDOFF.md	Rewrite assumptions that say 4 ultrasonic/front fixed ToF and required IMU as current facts	HIGH

14. Mapping Data Model

Recommended internal data model for the scanning ToF:

```
ToFScanPoint
-----
timestamp_ms: integer
seq: integer
angle_deg: float
distance_mm: float
valid: boolean

RobotPose
-----
timestamp_ms: integer
x_m: float
y_m: float
yaw_rad: float
source: ODOM / SLAM / UNKNOWN

MapPoint
-----
x_m: float
y_m: float
source_seq: integer
```

The mapper consumes ToFScanPoint + RobotPose. It ray-casts free space and marks obstacle endpoints. The output is an occupancy grid using the existing convention: 0 free, 100 occupied, -1 unknown.

15. Safety and Authority Rules

- Exactly one layer may be the movement authority at a time.
- Keep the Rover's 1-second command watchdog enabled.
- Map invalid sensor values as invalid, not as very large clear distances.
- Do not let missing ToF data automatically imply a clear path.
- Do not enable autonomous exploration until the connection-loss behavior is tested.
- Emergency stop must stop the actuator path independently of normal navigation commands.
- Software must distinguish normal STOP from EMERGENCY_STOP.
- During initial tests, use the low Rover PWM already selected for mapping/exploration.

The team's existing safety state model already includes DISCONNECTED, CONNECTING, IDLE, MANUAL, AUTONOMOUS, NAVIGATING, EXPLORING, EMERGENCY_STOP and FAULT. Reuse these states rather than creating a second state machine in the frontend.

16. Integration Test Plan

Stage	Test	Pass condition
1	Brain network connection	Backend reaches Brain IP and receives heartbeat/telemetry
2	ToF scan	Angles progress 30→150→30 and distance values are valid
3	Ultrasonic front	Front distance appears in normalized mm; disconnected channels become zero
4	Telemetry WebSocket	Frontend receives sensor_telemetry with monotonic sequence numbers
5	Live scan UI	A polar/radar scan overlay follows the servo angle
6	Manual command path	STOP/FORWARD/LEFT/RIGHT reaches Rover correctly
7	Failsafe	Removing command stream causes Rover to stop
8	Map local scan	Stationary rover produces correct obstacle points
9	Pose integration	Map points remain correctly located as rover moves
10	Occupancy grid	MapPage renders /map updates without frontend changes
11	Autonomous mode	Only one navigation authority publishes movement

Stage	Test	Pass condition
12	Connection loss	UI shows disconnected/stale and rover stops safely

17. What Is Already Proven vs What Is Not

Item	Current status
Motor direction mapping	PROVEN in standalone testing
Rover default PWM 80	Configured for current low-speed prototype
Brain→Rover UART	PROVEN; Brain commands are received
Servo movement	PROVEN; continuous scan logic now uses 2° steps
VL53L0X measurement	PROVEN; real readings such as MAP,118,38 observed
Ultrasonic full set	NOT COMPLETE; only the front sensor is currently being used in testing
ESP32→laptop network telemetry	NOT YET IMPLEMENTED in current Brain firmware
ROS2 hardware bridge	NOT YET CONNECTED to actual ESP32 protocol
Reliable global rover pose	NOT AVAILABLE from current sensor set
Persistent global occupancy map while moving	NOT RELIABLE until pose/odometry is solved
Existing React map UI	ALREADY IMPLEMENTED
Existing backend WebSocket architecture	ALREADY IMPLEMENTED
Existing ROS2 provider abstraction	ALREADY IMPLEMENTED

18. Recommended Implementation Order

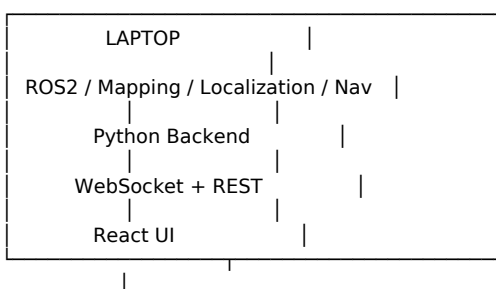
Do not ask the software team to change everything at once. Use this order:

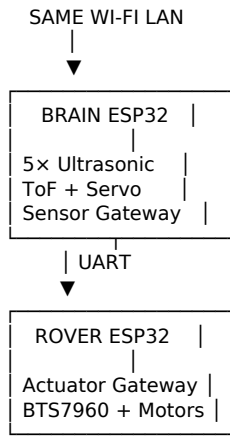
- Step 1: Add the ESP32Provider in the Python backend and define the Brain network JSON protocol.
- Step 2: Modify the Brain firmware only enough to expose the current sensor stream over Wi-Fi.
- Step 3: Confirm the backend receives ToF angle/distance and ultrasonic data.
- Step 4: Add a live scan event and display it in the existing React dashboard.
- Step 5: Produce a local polar/Cartesian point cloud from the ToF scan.
- Step 6: Add rover pose when a reliable source is available; only then build a persistent global map.
- Step 7: Integrate occupancy-grid publishing into the existing map contract.
- Step 8: Decide whether Brain autonomy or ROS2 autonomy is the sole movement authority.
- Step 9: If ROS2 autonomy is selected, implement the actuator velocity protocol and safety interlocks.
- Step 10: Run the physical integration test plan from sensor → map → navigation → exploration.

This order minimizes risk: first prove communication, then prove data, then prove visualization, then prove mapping, and only after that enable higher-level autonomy.

19. Final Target Architecture

ARACHNID FINAL SYSTEM





Data upward:
Sensors → Brain → Wi-Fi → Backend → ROS2/Map → React

Commands downward:
React/ROS2 → Backend → Wi-Fi/UART gateway → Rover → Motors

For the current prototype, the Brain may remain the navigation authority while the laptop software observes and maps. For the final autonomous architecture, choose a single navigation authority and make the interfaces explicit.

20. Software Team Handoff Checklist

- ☐ Update hardware.yaml to 5 ultrasonics and actual ToF model.
- ☐ Add ToF scan angle/distance to the telemetry schema.
- ☐ Add ESP32Provider or equivalent direct hardware gateway.
- ☐ Normalize cm→mm and 999→-1.
- ☐ Add Brain IP/port configuration through environment/config, not frontend hard-coding.
- ☐ Keep React connected only to the backend.
- ☐ Add tof_scan WebSocket event.
- ☐ Keep MapPage/LiveMap unless an optional scan overlay is desired.
- ☐ Decide and document navigation authority.
- ☐ Do not enable global SLAM until rover pose is available.
- ☐ Test connection-loss and command timeout.
- ☐ Update ROS2 `HARDWARE HANDOFF.md` so it matches the actual two-ESP32 architecture.

Bottom line: the team's software does not need to be rebuilt. The integration job is to create a well-defined ESP32 network gateway, normalize the sensor stream into the team's existing contracts, add the rotating-ToF scan model, and solve the pose source before claiming a true persistent 2D map.